

Apigee Global Gateway Traffic & Billing Governance

Environment-level API traffic governance for an existing 200–400 proxy estate

Document item	Value
Document ID	PR-API-0043
Prepared by	Arjun Krishna — API Engineer / Business Analyst
Date	September 2026
Implementation	Apigee X / compatible Apigee hybrid architecture
Repository	apigee-global-gateway-governance
Version	1.0 — Implementation Report

Source requirements: BRD-API-0043, Global Gateway Traffic & Billing Governance, Version 2.0; and BRD-API-0042, API Cost Optimization & Traffic Governance System, Version 1.0 Draft. Both are dated September 2026.

Implementation focus: zero-touch environment governance using a PreProxy Flow Hook, reusable Shared Flow, encrypted environment KVM, targeted per-proxy SpikeArrest, and Apigee Analytics. Backend services remain unchanged.

1. Executive Summary

The source BRD describes an Apigee environment with approximately 200–400 active API proxies. Those proxies already have project-specific quota, Spike Arrest, validation, and security controls. The business problem is aggregate gateway traffic and unexpected cloud billing exposure. The requested architecture is zero-touch: existing proxy deployments should not need to be edited or redeployed.

The final implementation uses a PreProxy Flow Hook to invoke a centralized Shared Flow for every proxy in the environment. The Shared Flow reads an environment KVM, identifies the current proxy, checks a bypass list and—only when explicitly configured and enforcement is enabled—applies a targeted SpikeArrest rate. Apigee Analytics supplies the usage and reporting layer.

Technical refinement: the original BRD's single native global Quota counter is not represented as implemented. Current Apigee documentation says Quota counters are scoped to API proxies and are not distributed among API proxies. The finished project therefore uses Analytics for aggregate observation and targeted SpikeArrest for runtime control.

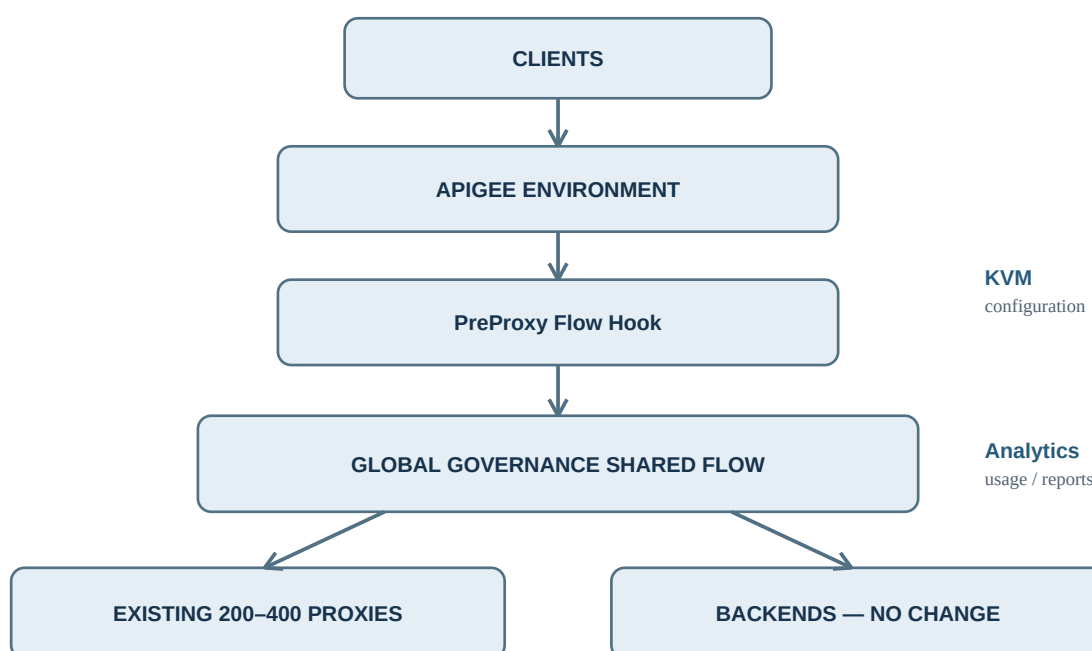


Figure 1. Final implementation architecture.

2. Requirements Baseline

BRD-API-0043 defines the global-gateway problem, zero-touch implementation, early traffic termination and centralized threshold management. It specifies four functional requirements: PreProxy Flow Hook integration, global billing quota enforcement, centralized KVM configuration, and a whitelist/bypass mechanism. It also specifies low overhead, zero-regression deployment, high availability, budget threshold alerts, proxy-level attribution and Trace diagnostics.

BRD-API-0042 is the earlier proxy-centric governance design. It adds tiered quota, Spike Arrest, KVM configuration, standardized 429 responses, response caching, latency/scalability targets and proactive alerting. The newer global-gateway design is the primary implementation scope.

Req	Source expectation	Final implementation	Disposition
FR-01	PreProxy Flow Hook + Shared Flow	Environment hook invokes global-governance Shared Flow.	Implemented
FR-02	One global billing quota	Analytics for aggregate observation + targeted per-proxy SpikeArrest.	Technically refined
FR-03	Encrypted central KVM	GLOBAL_GATEWAY_GOVERNANCE / config JSON.	Implemented
FR-04	Whitelist/bypass	Exact proxy names in bypassProxies.	Implemented
NFR	Low overhead / zero regression	Lightweight KVM + JS; no proxy edits; staged rollout.	Designed; benchmark in target org
Monitoring	Thresholds, attribution, Trace	Analytics queries/reports + governance variables.	Framework implemented

3. Existing System and Problem

Area	Current state
API estate	Approximately 200–400 active API proxies.
Proxy controls	Project-specific quota, Spike Arrest, validation and security policies already exist.
Business problem	Aggregate API volume is higher than expected and associated with increased cloud billing.
Operational risk	Editing hundreds of production proxies creates deployment and regression risk.
Backend	Backend services should remain unchanged.

The solution therefore adds a platform-level governance layer above the existing proxy logic rather than replacing individual API controls.

The key architectural requirement is that governance should be centralized while preserving the existing behavior of individual APIs. This is why the project does not require FlowCallout additions to each proxy.

4. Detailed Architecture

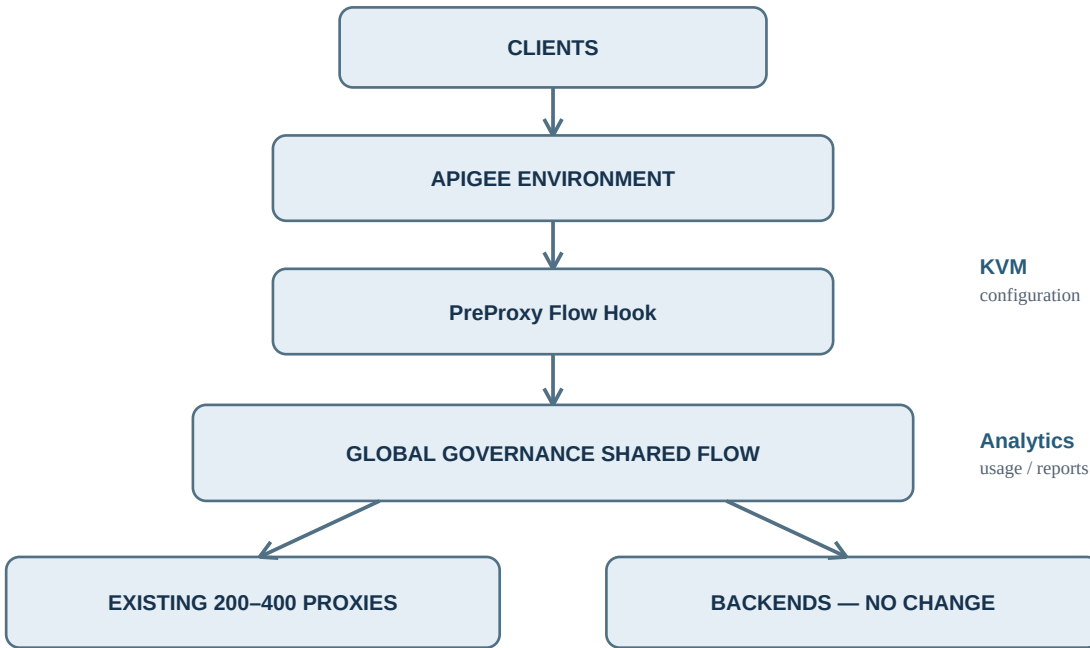


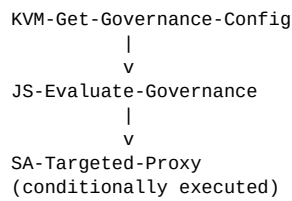
Figure 2. Request path and control plane.

Component	Purpose	Artifact
PreProxy Flow Hook	Environment-wide attachment point.	PreProxyFlowHook
Shared Flow	Reusable governance execution layer.	global-gateway-governance
KVM	Centralized configuration/control plane.	GLOBAL_GATEWAY_GOVERNANCE
JavaScript	Parses config and determines decision.	evaluate-governance.js
SpikeArrest	Targeted runtime rate control.	SA-Targeted-Proxy
Analytics	Usage, TPS, latency, errors and reports.	Custom Reports / API

Request lifecycle: Client → Apigee environment → PreProxy Flow Hook → Governance Shared Flow → existing API proxy → backend. KVM and Analytics sit beside this path as configuration and observability capabilities.

5. Shared Flow Implementation

The repository uses a standard Shared Flow bundle containing a bundle descriptor, policies, JavaScript resources and shared-flow execution definition.



The final execution condition is intentionally restrictive: SpikeArrest runs only when ``governance.enforce`` is true and ``governance.bypass`` is false. The JavaScript also requires the current proxy to have an explicit entry in ``throttleProxies``.

```
<Step>
  <Name>KVM-Get-Governance-Config</Name>
</Step>
<Step>
  <Name>JS-Evaluate-Governance</Name>
</Step>
<Step>
  <Condition>
    governance.enforce = "true"
    AND governance.bypass = "false"
  </Condition>
  <Name>SA-Targeted-Proxy</Name>
</Step>
```

6. KVM Configuration and Decision Engine

The environment KVM is `GLOBAL_GATEWAY_GOVERNANCE`. Its `config` entry contains JSON so operational thresholds can change without changing the Shared Flow code.

```
{
  "mode": "monitor",
  "throttleProxies": {
    "orders-api": "200ps",
    "catalog-api": "150ps"
  },
  "bypassProxies": [
    "payment-api",
    "authentication-api",
    "health-api"
  ]
}
```

The decision engine writes `governance.mode`, `governance.proxy`, `governance.bypass`, `governance.targeted`, `governance.rate`, `governance.rate.valid`, `governance.enforce`, `governance.decision`, `governance.config.valid` and `governance.config.error`. These variables are useful for Trace and diagnostics.

The engine supports three modes: **off** disables enforcement, **monitor** evaluates without throttling, and **enforce** throttles only explicitly configured proxies. A bypass entry wins over a throttle entry.

Fail-safe behavior: missing or invalid KVM configuration, unsupported mode, and invalid proxy rates fall back to monitor-only behavior. This prevents a malformed governance configuration from becoming an accidental environment-wide throttle.

7. Targeted SpikeArrest

SpikeArrest is used for short-term traffic shaping. It is not treated as a monthly business-consumption ledger. The policy uses the dynamic rate from the governance decision engine and identifies the proxy for rate-bucket isolation.

```
<SpikeArrest
  continueOnError="false"
  enabled="true"
  name="SA-Targeted-Proxy">
  <Identifier ref="governance.proxy"/>
  <Rate ref="governance.rate">100ps</Rate>
  <UseEffectiveCount>true</UseEffectiveCount>
</SpikeArrest>
```

Proxy	Rate	Mode	Decision
orders-api	200ps	enforce	THROTTLE
catalog-api	150ps	enforce	THROTTLE
search-api	—	enforce	MONITOR
payment-api	200ps	enforce + bypass	BYPASS

When the targeted rate is exceeded, SpikeArrest can reject excess requests with HTTP 429. Existing proxy-level quota and Spike Arrest policies remain active, so this is an additional governance layer rather than a replacement.

8. Why the Global Quota Requirement Was Refined

BRD-API-0043 FR-02 specifies a global Quota counter aggregating calls across all proxies. Current Apigee Quota documentation explicitly states that quota counters are scoped to the API proxy and are not distributed among API proxies. A standard Quota policy in a Shared Flow should therefore not be documented as one atomic counter for an entire 200–400-proxy environment.

Business requirement	Technical implementation
Global traffic visibility	Apigee Analytics grouped by proxy and time.
Runtime traffic control	Targeted per-proxy SpikeArrest.
Zero-touch deployment	PreProxy Flow Hook.
Dynamic thresholds	Environment KVM.
Critical-service protection	Bypass list.
Cost analysis	Correlate Apigee usage with Google Cloud Billing / FinOps.

This refinement preserves the business objective while matching documented Apigee policy scope. It also reduces the blast radius of enforcement because a noisy API can be targeted without intentionally putting unrelated APIs into the same runtime rate bucket.

9. Deployment Procedure

1. Clone the repository and configure ``.env`` with organization and environment.
2. Run ``.scripts/validate.sh``.
3. Create the environment KVM.
4. Upload monitor-mode configuration.
5. Import and deploy the Shared Flow.
6. Inspect existing Flow Hooks and confirm ownership.
7. Attach to PreProxyFlowHook only after change approval.
8. Exercise representative APIs in monitor mode.
9. Use Analytics and Trace to identify high-volume proxies.
10. Add one non-critical proxy to ``.throttleProxies``.
11. Switch to enforce mode and perform a controlled 429 test.
12. Expand gradually and retain bypass entries for approved critical services.

The repository intentionally does not attach the Flow Hook during bootstrap because Flow Hooks have environment-wide impact. The attach script requires explicit confirmation.

10. GitHub Repository

Path	Purpose
sharedflowbundle/	Deployable Apigee Shared Flow.
config/	KVM and Analytics configuration templates.
scripts/	Validation, packaging, import, deployment and operational controls.
tools/	Python project/config validators.
tests/	JavaScript decision-engine tests.
docs/	Architecture, runbook and troubleshooting.
.github/workflows/	GitHub Actions validation.
README.md	Detailed setup and operational guide.

```
./scripts/validate.sh
./scripts/package-sharedflow.sh

./scripts/create-kvm.sh
CONFIG_FILE=config/governance-config.example.json ./scripts/put-kvm-config.sh

./scripts/import-sharedflow.sh
./scripts/deploy-sharedflow.sh

./scripts/get-flow-hooks.sh
./scripts/attach-flow-hook.sh

./scripts/set-mode.sh monitor config/your-config.json
./scripts/set-mode.sh off config/your-config.json
./scripts/detach-flow-hook.sh
```

The GitHub Actions workflow validates XML, configuration, shell syntax, JavaScript decision logic and Shared Flow packaging on pushes and pull requests. It does not automatically deploy to production.

11. Testing Strategy

Level	Test	Expected result
Static	Parse all bundle XML.	No malformed XML.
Config	Validate modes, rates and lists.	Invalid config rejected.
Unit	Monitor with configured proxy.	MONITOR.
Unit	Enforce with explicit proxy.	THROTTLE.
Unit	Proxy not configured.	MONITOR.
Unit	Bypass + throttle overlap.	BYPASS.
Unit	Invalid JSON/rate.	Fail-open to monitor.
Environment	Flow Hook Trace.	Governance variables visible.
Environment	Controlled burst.	Expected 429s for targeted proxy.
Regression	Representative existing APIs.	Existing logic unaffected.
Analytics	Before/after query.	Traffic and latency attributable by proxy.

The repository tests the decision engine automatically. Production readiness still requires testing in the actual Apigee organization because IAM, analytics entitlements, existing Flow Hook ownership, proxy behavior and traffic characteristics are environment-specific.

12. Monitoring and Reporting

Report	Metric / dimension	Purpose
Gateway usage by proxy	message_count + apiproxy	Identify highest-volume APIs.
TPS by proxy	tps + apiproxy	Identify high-rate APIs.
Latency	response time + apiproxy	Check application/governance impact.
Errors	4xx/5xx + apiproxy	Identify failures.
429 analysis	status/policy errors + apiproxy	Measure throttling.
Before vs after	traffic + TPS + errors	Assess governance effect.
Governance decisions	custom captured data when enabled	Separate MONITOR/THROTTLE/BYPASS.

Apigee Custom Reports can use metrics, dimensions and filters, and can use custom data collected with DataCapture. Report jobs can be used for larger time ranges or more complex analytics. The project includes an example Analytics query and a script for submitting it.

The project does not calculate the exact cloud invoice. For billing governance, combine Apigee traffic/usage evidence with Google Cloud Billing and FinOps data.

13. Security, Reliability and Rollback

Concern	Control
Secrets	No credentials in repository; <code>`.env`</code> ignored.
KVM	Environment-scoped centralized configuration.
Bad configuration	Decision engine falls back to monitor-only.
Critical APIs	Explicit bypass list.
Blast radius	Flow Hook attachment is separate and confirmed.
Rollback	Set mode to <code>`off`</code> or detach Flow Hook.
Existing security	Existing proxy authentication/validation remains unchanged.
Change control	Monitor first; enforce gradually.

A PreProxy Flow Hook is an environment-wide control. Production changes should therefore be treated as platform-level changes with review, approval, rollback planning and observation windows.

14. Limitations and Assumptions

- Flow Hook management requires appropriate Apigee permissions and has environment-wide blast radius.
- Only one Shared Flow can be attached to a particular Flow Hook point; an existing hook must be reviewed before attachment.
- KVM values are cached according to the policy configuration, so configuration changes are not necessarily instantaneous.
- SpikeArrest is a rate/burst control, not a monthly business quota.
- Exact cloud billing must be obtained from Google Cloud Billing / FinOps data.
- Analytics features depend on the Apigee subscription and analytics entitlement.
- Proxy names are matched exactly; wildcard matching is intentionally not used in the initial version.
- Existing proxy policies remain active and may independently reject traffic.
- The BRD performance targets, including the under-15ms governance overhead target, require benchmarking in the target organization; this report does not claim that production benchmark is complete.

15. BRD Traceability

BRD area	Expectation	Implementation evidence
Zero-touch	No proxy modifications.	PreProxy Flow Hook; no FlowCallout additions.
Early termination	Reject before normal proxy/backend work where governance limit applies.	PreProxy governance + SpikeArrest.
Central threshold	Dynamic centralized configuration.	Environment KVM.
Bypass	Critical proxy protection.	bypassProxies.
Global quota	Aggregate counter.	Refined to Analytics observation + targeted runtime control due documented Quota scope.
Monitoring	Proxy attribution and diagnostics.	Analytics dimensions + governance variables.
Alerting	Budget/usage visibility.	Analytics/reporting framework; alert integration is a future operational extension.

16. Future Enhancements

Automated alerts: connect Analytics/report outputs to Cloud Monitoring, email or collaboration channels.

FinOps correlation: join Apigee usage with Google Cloud Billing exports.

Administration portal: provide an approved interface for changing proxy rates.

Consumer-aware controls: add application/consumer dimensions where business requirements justify them.

Central dashboard: visualize traffic, throttling, latency and cost.

Anomaly detection: identify unusual traffic growth and produce candidates for review without automatic enforcement.

17. Conclusion

The completed project provides a practical, version-controlled Apigee implementation for environment-level API traffic governance. Its defining feature is zero-touch integration: the governance logic lives outside the 200–400 existing proxy bundles and is attached through the environment's PreProxy Flow Hook.

The project separates observation from enforcement. Analytics provides gateway-wide evidence and proxy-level attribution, while targeted SpikeArrest provides controlled runtime traffic shaping for selected APIs. KVM provides the operational configuration plane, and explicit bypass rules protect approved critical services.

The native global Quota requirement is intentionally refined rather than misrepresented. Current Apigee documentation states that Quota counters are scoped to API proxies. The implementation therefore achieves the business goal through documented, testable mechanisms: Flow Hook, Shared Flow, KVM, targeted SpikeArrest and Analytics.

Production sign-off should require: staged rollout, actual latency measurement, controlled 429 testing, regression testing of representative APIs, analytics verification, security review and correlation against real cloud billing data.

18. References

BRD-API-0043: Global Gateway Traffic & Billing Governance, Version 2.0, September 2026.

BRD-API-0042: API Cost Optimization & Traffic Governance System, Version 1.0 Draft, September 2026.

Google Cloud / Apigee: Flow Hooks — <https://docs.cloud.google.com/apigee/docs/api-platform/fundamentals/flow-hooks>

Google Cloud / Apigee: Shared Flows — <https://docs.cloud.google.com/apigee/docs/api-platform/fundamentals/shared-flows>

Google Cloud / Apigee: Quota policy — <https://docs.cloud.google.com/apigee/docs/api-platform/reference/policies/quota-policy>

Google Cloud / Apigee: SpikeArrest policy — <https://docs.cloud.google.com/apigee/docs/api-platform/reference/policies/spike-arrest-policy>

Google Cloud / Apigee: Custom reports — <https://docs.cloud.google.com/apigee/docs/api-platform/analytics/create-custom-reports>

Google Cloud / Apigee: Asynchronous custom reports API —
<https://docs.cloud.google.com/apigee/docs/api-platform/analytics/asynch-reports-api>

Google Cloud / Apigee: Analytics metrics reference — <https://docs.cloud.google.com/apigee/docs/api-platform/analytics/analytics-reference>